National University of Computer and Emerging Sciences



Lab Manual 05
Computer Organization
and Assembly Language

Course Instructor	Mr. Salman Mubarik
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Fall 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Objectives: 3

Activities: 3

Activity 1 (Theoretical Concepts): 3

1. Subroutines 3
2. Stack 3
3. Stack Registers 3
4. PUSH Instruction 4
5. POP Instruction 4
6. CALL Instruction 4
7. RET Instruction 4
8. Passing Parameters Through the Stack 5
9. Why Use BP Instead of SP? 5

Activity 2 (Assembly Language Tasks): 5

1) Merge Two Sorted Arrays 5
2) Interleave Two Arrays 6
3) Count Trailing Zeros 6
4) Split Packed Nibbles 7

Objectives:

- Understand the concept, purpose and use of subroutines/functions
- Understand and use the stack for temporary storage and parameter passing.

Activities:

Activity 1 (Theoretical Concepts):

1. *Subroutines*

   A subroutine is a reusable block of instructions designed to perform a specific task. Instead of writing the same instructions repeatedly, a program can call the subroutine whenever that task is required.
   For example, instead of writing the instructions for addition several times:
   CALL AddNumbers
   The processor executes the AddNumbers subroutine and then returns to the instruction following the CALL.

   General structure

   ```
   CALL SubroutineName

   ; execution continues here after return

   SubroutineName:
   ; instructions
   RET
   ```

   A subroutine is similar to a function in high-level languages such as C/C++.

2. *Stack*

   A stack is a memory area used for temporary storage of data. The stack follows the LIFO — Last In, First Out principle.

   For example:

   ```
   PUSH 1111h
   PUSH 2222h
   PUSH 3333h
   ```

   The value pushed last (3333h) is removed first.

   ```
   POP AX    ; AX = 3333h
   POP BX    ; BX = 2222h
   POP CX    ; CX = 1111h
   ```

3. *Stack Registers*

   In 8086, stack operations primarily involve three registers:

Register	Purpose
SS	Stack Segment — identifies the memory segment containing the stack

SP	Stack Pointer — points to the top of the stack
BP	Base Pointer — commonly used to access parameters and local variables in a subroutine

The stack is addressed using SS:SP. The stack grows toward lower memory offsets. BP is particularly useful because memory operands using BP default to the stack segment (SS).

4. *PUSH Instruction*

PUSH places a word onto the stack. The stack pointer decreases by 2 for a 16-bit push.
Syntax:
PUSH source

Example:
MOV AX, 1234h
PUSH AX

5. *POP Instruction*

POP removes the value from the top of the stack and places it into the specified destination. The stack pointer increases by 2 for a 16-bit pop.
Syntax:
POP destination

Example:
PUSH AX
POP BX

6. *CALL Instruction*

CALL is used to invoke a subroutine.
Example:
CALL AddNumbers

For a near call, the processor saves the return address on the stack and transfers control to the subroutine. The return address is the address of the instruction immediately following the CALL.

7. *RET Instruction*

RET returns execution from a subroutine to the caller. A near RET retrieves the saved instruction pointer from the stack. Therefore, the stack must contain the correct return address when RET executes.

Example:
CALL MyFunction
; execution resumes here

MyFunction:
; function instructions
RET

Important
If a subroutine pushes additional values onto the stack, it must handle them properly before executing RET. For example, this is dangerous:

```
MyFunction:
    PUSH AX
    RET
```

because the top of the stack is now the saved AX, not the return address.

8. *Passing Parameters Through the Stack*

Parameters can be passed to a subroutine using the stack. For example, suppose we want:

Add(a,b)

The caller can push the parameters:

```
PUSH 5
PUSH 3
CALL Add
```

Inside the subroutine, the parameters can be accessed using BP.

9. *Why Use BP Instead of SP?*

Suppose a function has:

```
PUSH AX
PUSH BX
PUSH CX
```

SP keeps changing as values are pushed and popped. Therefore, using SP directly to access parameters can become difficult. Instead:

```
PUSH BP
MOV BP, SP
```

creates a fixed reference point. Then parameters can be accessed relative to BP:

```
MOV AX, [BP+4]
MOV BX, [BP+6]
```

This makes the subroutine easier to manage.

Activity 2 (Assembly Language Tasks):

1) *Merge Two Sorted Arrays*

Write a subroutine:
    MergeArrays
that merges two already sorted arrays of words into one sorted array.

Input
    A1: [2, 5, 9]
    A2: [1, 4, 7, 8]
Output
    Merged: [1, 2, 4, 5, 7, 8, 9]

Requirements

- Both input arrays are already sorted in ascending order.

- Do not sort the input arrays first.
- Compare elements from both arrays.
- Copy the smaller element to Merged.
- Continue until one array is exhausted.
- Copy the remaining elements of the other array.
- Implement the merging logic inside a subroutine.
- Pass required information to the subroutine through the stack.

2) *Interleave Two Arrays*

Write a subroutine:
InterleaveArrays
that reads two byte arrays A and B and creates an output array containing alternating elements.

Example
A: [10, 11, 12]
B: [20, 21, 22, 23]

Output:
[10, 20, 11, 21, 12, 22, 23]

Requirements
- Take the two arrays as input.
- Alternate elements from the two arrays.
- If one array finishes first, append the remaining elements of the other array.
- Use appropriate index/pointer registers.
- Implement the operation as a subroutine.

3) *Count Trailing Zeros*

Write a subroutine:
CountTrailingZeros
that counts the number of consecutive 0 bits at the right-hand side of a 16-bit number. The right-hand side means the least significant bit (LSB) side.

Example
AX = 0010 1000 0000 0000b
or:
AX = 2800h

The binary representation ends with:
000 0000 0000

Therefore:
Result = 11

Requirements
- Work with a 16-bit value.
- Examine the bits from the LSB toward the MSB.
- Count consecutive zero bits.
- Stop when a 1 bit is encountered.
- Return the count from the subroutine.

4) Split Packed Nibbles

Each byte in the array Packed[] contains two 4-bit values:
Write a subroutine:

SplitNibbles

that separates the upper and lower nibbles into two arrays.

Input

Packed: [0xAB, 0x3C, 0x7F]

Expected Output

HighNib: [0x0A, 0x03, 0x07]
LowNib: [0x0B, 0x0C, 0x0F]

Requirements

- Read each byte from Packed.
- Extract the upper 4 bits.
- Extract the lower 4 bits.
- Store each as a separate byte.
- Upper and lower nibble values must be zero-extended.
- Do not use rotation instructions.
- Implement the operation as a subroutine.

Hint

For a byte such as:

ABh = 1010 1011b
└──┘ └──┘
High Low

The desired result is:
HighNib = 0Ah
LowNib = 0Bh